

LeRobot v0.6.0 • 实操型参考手册

SmolVLA 教学手册

LIBERO 仿真评估 · Piper 真机微调 · 本地/云端推理

适用环境: Ubuntu 22.04 / AutoDL GPU / LeRobot 0.6.0 / Piper

执行路线

1. 步骤 1–8：在云服务器上完成 Conda、CMake、EGL、LeRobot 与 CUDA 环境准备。
2. 步骤 9–13：保持联网，下载 SmolVLA、SmolVLM2 和 LIBERO 资源。
3. 步骤 14–17：绑定 Piper 数据集与本地模型路径，然后切换到离线 EGL 模式。
4. 步骤 18–19：先评估 `smolvla_libero`，再用 Piper 数据微调 `smolvla_base`。
5. 步骤 20：把检查点下载到机器人电脑，运行本地同步 rollout。
6. 步骤 21–23：使用三个终端完成云端 Policy Server、SSH 隧道和本地 Robot Client 的异步推理链路。

阅读约定：灰底等宽字体区域均为原始命令。蓝色说明是操作解释，金色提示是容易出错的条件，红色提示涉及真机安全或链路风险。命令应按文档顺序逐段执行，不要一次性跨终端整页粘贴。

重要区别：步骤 18 评估的是下载好的 `smolvla_libero`；步骤 19 训练的是 `smolvla_base` + Piper 真机数据。当前命令没有使用 LIBERO 数据集训练新的 SmolVLA，因此不要把两者的模型和指标混在一起。

第一部分 云服务器环境与依赖准备

完成 Conda 环境激活、目录统一、编译工具链检查、EGL 组件安装与 LeRobot 0.6.0 依赖核验。

步骤 1 激活环境并建立统一目录

执行位置：云服务器（联网阶段）

本步目标：激活 lerobot Conda 环境，定义 pip、Hugging Face、模型、LIBERO assets、检查点和评估日志的统一位置，并预先创建目录。

这些 export 变量只对当前终端会话及其子进程生效。后续命令依赖这些变量，因此不要在执行中途随意关闭当前终端；如果重新开终端，需要从本步骤重新设置。mkdir -p 可以重复执行，已有目录不会被删除。

原始命令（保持不变）

```
conda activate lerobot
#设置环境变量
export PIP_INDEX_URL=https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple

export HF_HOME=/root/autodl-tmp/cache/huggingface
export HF_LEROBOT_HOME=/root/autodl-tmp/huggingface/lerobot

export SMOLVLA_LIBERO=/root/autodl-tmp/pretrained/smolvla_libero
export SMOLVLA_BASE=/root/autodl-tmp/pretrained/smolvla_base
export SMOLVLM_DIR=/root/autodl-tmp/cache/huggingface/manual_models/SmolVLM2-500M-Video-Instruct
export LIBERO_ASSETS=/root/autodl-tmp/libero/assets

mkdir -p \
  "$HF_HOME" \
  "$HF_LEROBOT_HOME" \
  "$SMOLVLA_LIBERO" \
  "$SMOLVLA_BASE" \
  "$SMOLVLM_DIR" \
  "$LIBERO_ASSETS" \
  /root/autodl-tmp/checkpoints \
  /root/autodl-tmp/eval_logs
```

完成标志：所有目录创建完成，终端没有出现 Permission denied；后续使用变量时不应展开为空字符串。

说明：本手册原样保留清华镜像地址。如果 pip 后续出现 404、连接失败或找不到本应存在的包，应先核对网络和镜像可用性，不要把依赖问题与镜像问题混在一起判断。

步骤 2 重装兼容版本的 CMake

执行位置：云服务器（联网阶段）

本步目标：清除可能干扰 CMake 的环境变量，并强制安装 3.29 以上、4.0 以下的二进制版本。

--force-reinstall 会覆盖当前环境中的 CMake；--no-cache-dir 避免复用损坏缓存；--only-binary=:all: 要求直接安装 wheel，从而避开本机源码编译。版本上限用于避免 CMake 4.x 与部分旧构建脚本不兼容。

原始命令（保持不变）

```
#重装 cmake
unset CMAKE_ROOT
unset CMAKE_POLICY_VERSION_MINIMUM

python -m pip install \
  --force-reinstall \
  --no-cache-dir \
  --only-binary=:all: \
  "cmake>=3.29,<4"
```

完成标志：pip 安装过程以 Successfully installed 结束，且没有残留 build failed 或 dependency conflict。

步骤 3 检查 Make、GCC 与 G++ 工具链

执行位置：云服务器

本步目标：确认构建 egl-probe 等本地扩展所需的 make、gcc、g++ 均可被当前 shell 找到。

command -v 负责显示可执行文件路径；版本命令确认程序能够正常启动。这里重点看“路径存在”和“版本可输出”，不能只看软件包是否曾经安装过。

原始命令（保持不变）

```
command -v make
command -v gcc
command -v g++
make --version
g++ --version
```

完成标志：五条检查均有正常输出，且没有 command not found。

注意：如果其中任一项为空或报错，后续 egl-probe 的失败通常只是结果，不是根因。应先修复系统编译工具链，再继续执行。

步骤 4 安装 EGL 设备探测包

执行位置：云服务器（联网阶段）

本步目标：确保优先使用 Conda 环境中的工具，然后安装 egl-probe 与 hf-egl-probe 1.0.2。

--no-build-isolation 表示构建时直接使用当前环境中的 CMake、编译器和 Python 依赖；--no-cache-dir 防止再次使用此前失败的 wheel。两个包都安装成功才算完成，hf-egl-probe 成功不代表 egl-probe 也成功。

原始命令（保持不变）

#安装两个 EGL probe 包

```
export PATH="$CONDA_PREFIX/bin:$PATH"
```

```
export PIP_INDEX_URL=https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple
```

```
python -m pip install \
  --no-build-isolation \
  --no-cache-dir \
  "egl-probe==1.0.2" \
  "hf-egl-probe==1.0.2"
```

完成标志：安装日志中两个包均显示安装成功，不再出现 cmake ..; make -j returned non-zero exit status。

说明：本步骤保留原执行顺序。如果 egl-probe 需要源码编译，应重点查看失败日志最后几十行，以区分 CMake、头文件、链接库或编译器问题。

步骤 5 检测可用的 EGL GPU 设备

执行位置：云服务器

本步目标：调用 egl-probe 枚举可用于无界面渲染的 GPU 设备，为 MuJoCo/LIBERO 的 headless EGL 渲染做验证。

该命令验证的是 EGL 渲染能力，不只是 CUDA 是否可用。nvidia-smi 正常、PyTorch 能看到 GPU，也可能因为驱动能力或 EGL 库问题而探测失败。

原始命令（保持不变）

```
python -m egl_probe.get_available_devices
```

完成标志：输出中出现可用设备编号，并且设备测试通过；若出现 EGL cannot run on this device，不能把该结果当作成功。

步骤 6 安装系统 EGL/OpenGL 运行库

执行位置：云服务器（需要 root 权限）

本步目标：更新 apt 索引并安装无界面渲染所需的 EGL、GLVND、GLES2 与 OpenGL 运行库。

apt-get install -y 会自动确认安装。hash -r 用于清空 shell 对命令路径的缓存，使新安装的程序或库相关命令在当前终端中立即按新状态解析。

原始命令（保持不变）

```
#安装 EGL 渲染
apt-get update
apt-get install -y \
    libegl1 \
    libglvnd0 \
    libgles2 \
    libgl1
hash -r
```

完成标志：四个系统包安装完成，没有 unmet dependencies；hash -r 无报错返回。

步骤 7 安装 LeRobot 0.6.0 全套依赖

执行位置：云服务器（联网阶段）

本步目标：一次安装训练、SmolVLA、LIBERO 和异步推理所需的 LeRobot 0.6.0 extras。

方括号中的 training、smolvla、libero、async 会拉取不同功能组的附加依赖；==0.6.0 固定 LeRobot 主版本，避免命令行参数随新版本变化。pip 可能升级或降级已有依赖，应以本步骤之后的环境检查为准。

原始命令（保持不变）

```
#安装 lerobot 依赖
python -m pip install \
    "lerobot[training,smolvla,libero,async]==0.6.0"
```

完成标志：安装完成并显示 lerobot 0.6.0，未出现 ERROR 或构建失败。

步骤 8 检查 Python 包、CUDA 与依赖一致性

执行位置：云服务器

本步目标：打印关键包版本、CUDA 可用性和 GPU 名称，再用 pip check 检查依赖冲突。

Python here-document 会逐项读取已安装版本；未安装的包显示“未安装”。随后检查 `torch.cuda.is_available()` 和第 0 块 GPU 名称。pip check 的理想结果是 No broken requirements found。

原始命令（保持不变）

#检查环境

```
python - <<'PY'
import importlib.metadata as md
import torch
packages = [
    "lerobot",
    "torch",
    "torchvision",
    "transformers",
    "accelerate",
    "datasets",
    "num2words",
    "grpcio",
    "protobuf",
    "scipy",
    "matplotlib",
    "hf-libero",
    "egl-probe",
    "hf-egl-probe",
]

for name in packages:
    try:
        print(f"{name:18s}: {md.version(name)}")
    except md.PackageNotFoundError:
        print(f"{name:18s}: 未安装")

print("CUDA:", torch.cuda.is_available())
print("GPU:", torch.cuda.get_device_name(0))
PY
python -m pip check
```

完成标志：关键包均有版本号，CUDA 为 True，GPU 名称正确，pip check 未报告冲突。

注意：代码会无条件读取第 0 块 GPU。如果 CUDA 不可用，它会在 GPU 名称处直接报错。因此执行前应确保 AutoDL 已分配 GPU、驱动可见且当前终端没有屏蔽 CUDA 设备。

第二部分 在线下载模型与 LIBERO 资源

暂时退出离线模式，使用镜像下载仿真策略、基础模型、视觉语言主干和 LIBERO assets。

步骤 9 退出离线模式并配置 Hugging Face 镜像

执行位置：云服务器（联网下载阶段）

本步目标：临时允许 Hugging Face、Transformers 与 Datasets 联网，并配置镜像、缓存位置、下载超时及单文件下载方式。

unset 三个离线变量后，下载命令才能访问远端。HF_ENDPOINT 指向镜像；HF_HUB_DISABLE_XET=1 禁用 Xet 下载路径；600 秒超时适合大文件和不稳定链路。完成全部下载后，后文会重新切换回离线模式。

原始命令（保持不变）

#配置国内网络镜像

```
unset HF_HUB_OFFLINE
unset TRANSFORMERS_OFFLINE
unset HF_DATASETS_OFFLINE
```

```
export HF_ENDPOINT=https://hf-mirror.com
export HF_HOME=/root/autodl-tmp/cache/huggingface
export HF_HUB_DISABLE_XET=1
export HF_HUB_DISABLE_UPDATE_CHECK=1
export HF_HUB_DOWNLOAD_TIMEOUT=600
```

完成标志：环境变量设置完成，后续 hf download 能开始解析仓库并写入指定目录。

步骤 10 下载 SmolVLA-LIBERO 仿真策略

执行位置：云服务器（联网下载阶段）

本步目标：把已经针对 LIBERO 使用的策略 lerobot/smolvla_libero 下载到 SMOLVLA_LIBERO 本地目录。

该目录在后续 LIBERO 评估中作为 --policy.path 使用。它与 smolvla_base 用途不同：前者用于仿真评估，后者用于在自己的 Piper 数据上继续微调。

原始命令（保持不变）

#下载仿真策略

```
hf download \
```



```
lerobot/smolvla_libero \
--local-dir "$SMOLVLA_LIBERO"
```

完成标志：下载完成，目标目录内至少可见策略配置、预处理配置和模型权重文件。

步骤 11 下载 SmolVLA Base 基础模型

执行位置：云服务器（联网下载阶段）

本步目标：下载 SmolVLA 基础检查点，为 Piper 数据微调提供初始化权重。

训练命令会把 SMOLVLA_BASE 作为 --policy.path。该模型是通用基础模型，直接在真机任务上运行通常不如经过本任务数据微调后的检查点。

原始命令（保持不变）

```
#下载基础模型
hf download \
  lerobot/smolvla_base \
  --local-dir "$SMOLVLA_BASE"
```

完成标志：SMOLVLA_BASE 目录完整，训练时不需要再次联网拉取主策略文件。

步骤 12 下载 SmolVLM2 视觉语言主干

执行位置：云服务器（联网下载阶段）

本步目标：下载 SmolVLA 使用的 SmolVLM2-500M-Video-Instruct，并排除本流程不需要的 ONNX 文件。

--exclude onnx/* 可以减少下载体积。后文会把两个 SmolVLA 策略目录内指向在线模型 ID 的配置值替换为该本地绝对路径，从而支持断网加载。

原始命令（保持不变）

```
#下载视觉语言主干
hf download \
  HuggingFaceTB/SmolVLM2-500M-Video-Instruct \
  --local-dir "$SMOLVLM_DIR" \
  --exclude "onnx/*"
```

完成标志：SMOLVLM_DIR 中存在 config、tokenizer 和 safetensors 等模型文件，且大文件下载完整。

步骤 13 下载 LIBERO assets

执行位置：云服务器（联网下载阶段）

本步目标：以数据集类型下载 LIBERO 场景、物体和纹理资源，并限制为单 worker 以降低镜像连接压力。

--repo-type dataset 指明远端是数据集仓库；--max-workers 1 用串行下载换取更稳定的网络行为。该目录随后会链接到 LIBERO 默认缓存位置。

原始命令（保持不变）

#下载 libero 资源

```
hf download lerobot/libero-assets \
```

```
--repo-type dataset \
```

```
--local-dir "$LIBERO_ASSETS" \
```

```
--max-workers 1 #最开始不加这个命令，报 429 联网限流加上重新跑，会接着前面下载
```

完成标志：LIBERO_ASSETS 目录包含完整资源，不再需要运行时从外网补下载。

第三部分 本地资源绑定与离线运行配置

指定 Piper 数据集，把模型配置改为本地 VLM 路径，初始化 LIBERO，并切换到完全离线的 EGL 渲染模式。

步骤 14 指定 Piper 真机数据集

执行位置：云服务器

本步目标：把本地 Piper LeRobot 数据集根目录保存到 PIPER_DATASET，并用 meta/info.json 做最小存在性检查。

info.json 是 LeRobot 数据集的核心元数据入口。命令只有在文件存在时才打印“Piper dataset 已存在”；没有输出通常意味着路径写错、数据没有上传完整或目录层级多了一层。

原始命令（保持不变）

#设置数据集

```
export PIPER_DATASET=/root/piper_record_test
```

```
test -f "$PIPER_DATASET/meta/info.json" && echo "Piper dataset 已存在"
```

完成标志：终端明确打印 Piper dataset 已存在。

注意：本步骤指定的是 Piper 真机数据集 `/root/piper_record_test`，不是 LIBERO 数据集。后面的训练因此是 Piper 任务微调。

步骤 15 把 VLM 在线模型 ID 改为本地路径

执行位置：云服务器

本步目标：递归扫描两个 SmolVLA 目录中的三个 JSON 配置，把精确匹配的 SmolVLM2 在线 ID 替换为本地 SMOLVLM_DIR 路径。

脚本只替换值完全等于旧模型 ID 的字段，不会模糊修改其他字符串。每个 JSON 首次处理前会生成 `.online.bak` 备份；重复执行时不会覆盖已有备份。`ensure_ascii=False` 保持中文或非 ASCII 字符可读。

原始命令（保持不变）

#把 vlm 路径改成本地

```
python - <<'PY'
```

```
import json
```

```
import shutil
```

```
from pathlib import Path
```

```
roots = [
```

```
    Path("/root/autodl-tmp/pretrained/smolvla_libero"),
```

```
    Path("/root/autodl-tmp/pretrained/smolvla_base"),
```

```
]
```

```
old = "HuggingFaceTB/SmolVLM2-500M-Video-Instruct"
```

```
new = "/root/autodl-tmp/cache/huggingface/manual_models/SmolVLM2-500M-Video-Instruct"
```

```
def replace_value(value):
```

```
    if isinstance(value, dict):
```

```
        return {k: replace_value(v) for k, v in value.items()}
```

```
    if isinstance(value, list):
```

```
        return [replace_value(v) for v in value]
```

```
    if value == old:
```

```
        return new
```

```
    return value
```

```
for root in roots:
```

```
    for filename in [
```

```
        "config.json",
```

```
        "policy_preprocessor.json",
```

```
        "train_config.json",
```

```
    ]:
```

```
        path = root / filename
```

```
        if not path.exists():
```

```
            continue
```

```
        backup = path.with_suffix(path.suffix + ".online.bak")
```

```

if not backup.exists():
    shutil.copy2(path, backup)

data = json.loads(path.read_text(encoding="utf-8"))
updated = replace_value(data)
path.write_text(
    json.dumps(updated, ensure_ascii=False, indent=2) + "\n",
    encoding="utf-8",
)
print("已处理:", path)
PY

```

完成标志：终端为实际存在的配置打印“已处理”，且 JSON 仍可正常解析；离线加载时不再尝试访问 Hugging Face。

步骤 16 初始化 LIBERO 并建立 assets 软链接

执行位置：云服务器

本步目标：完成 LIBERO 首次导入初始化，并把下载好的 assets 链接到 /root/.cache/libero/assets。

条件判断保证仅在目标既不存在、也不是软链接时创建链接，避免无意覆盖已有内容。最后 readlink -f 会显示软链接最终指向的真实目录。

原始命令（保持不变）

```

#初始化 libero
printf '\n\n' | python -c "import libero.libero"
mkdir -p /root/.cache/libero

if [ ! -e /root/.cache/libero/assets ] && [ ! -L /root/.cache/libero/assets ]; then
    ln -s "$LIBERO_ASSETS" /root/.cache/libero/assets
fi

readlink -f /root/.cache/libero/assets

```

完成标志：readlink -f 输出 /root/autodl-tmp/libero/assets，且导入 LIBERO 不再要求在线下载资源。

注意：如果 /root/.cache/libero/assets 已存在但指向错误位置，本段不会自动覆盖它。此时应先人工确认现有对象的类型和指向，再决定如何处理。

步骤 17 切换到完全离线与无界面 EGL 模式

执行位置：云服务器（离线运行阶段）

本步目标：固定 Hugging Face 缓存位置，并让 MuJoCo/PyOpenGL 使用 EGL。

三个 *_OFFLINE=1 变量用于验证所有模型与数据都来自本地；MUJOCO_GL=egl 和

PYOPENGL_PLATFORM=egl 面向无显示器的云服务器；关闭 tokenizer 并行提示可减少多进程环境中的告警。

原始命令（保持不变）

#切换本地模式

```
export HF_HOME=/root/autodl-tmp/cache/huggingface
export HF_LEROBOT_HOME=/root/autodl-tmp/huggingface/lerobot
```

```
export HF_HUB_OFFLINE=1
export TRANSFORMERS_OFFLINE=1
export HF_DATASETS_OFFLINE=1
```

```
export MUJOCO_GL=egl
export PYOPENGL_PLATFORM=egl
export TOKENIZERS_PARALLELISM=false
```

完成标志：后续模型加载不再发起网络请求，LIBERO 能在无桌面环境中创建仿真。

第四部分 LIBERO 评估与 Piper 微调

先评估官方 SmoIVLA-LIBERO 策略，再使用 Piper 真机数据微调 SmoIVLA Base。两段流程的数据与策略用途不同。

步骤 18 运行四套件 LIBERO 标准评估

执行位置：云服务器 GPU

本步目标：使用本地 smolvla_libero 策略，以 relative 控制模式依次评估 Spatial、Object、Goal 和 LIBERO-Long 四个标准套件。

四个套件各含 10 个任务；--eval.n_episodes=10 表示每个任务运行 10 回合，因此完整评估共 400 回合。batch_size=1、max_parallel_tasks=1 和 use_async_envs=false 采用串行方式，速度较慢但更稳。rename_map 按“环境源键 → 策略目标键”把 image/image2 映射为 camera1/camera2；empty_cameras=1 为策略所需的额外视觉槽位添加被掩蔽的占位相机，不会改动动作键。

原始命令（保持不变）

#仿真

```
EVAL_ALL=/root/autodl-tmp/eval_logs/smolvla_trained_all
```

```
lerobot-eval \
  --output_dir="$EVAL_ALL" \
  --policy.path="$SMOLVLA_LIBERO" \
  --policy.device=cuda \
  --policy.empty_cameras=1 \
  --env.type=libero \
  --env.task=libero_spatial,libero_object,libero_goal,libero_10 \
  --env.control_mode=relative \
  --env.max_parallel_tasks=1 \
  --eval.batch_size=1 \
  --eval.n_episodes=10 \
  --eval.use_async_envs=false \
  --
  rename_map='{"observation.images.image":"observation.images.camera1","observation.images.image2":"observation.images.camera2"}'
```

完成标志：EVAL_ALL 目录生成评估日志与回放文件，终端/汇总结果出现各任务成功率及整体 pc_success; pc_success=100% 表示全部评估回合成功。

可以去云服务器/root/autodl-tmp/eval_logs/smolvla_trained_all 查看视频和准确率

注意：该输出目录名称是固定的 smolvla_trained_all。重复运行前应意识到新结果仍写入同一路径，整理实验时要避免把不同轮次混为一组。

步骤 19 在 Piper 数据上微调 SmolVLA Base

执行位置：云服务器 GPU

本步目标：使用 /root/piper_record_test 真机数据和 smolvla_base 初始化权重，训练 20,000 步并每 5,000 步保存一次检查点。

dataset.root 指向本地数据集，dataset.repo_id 提供数据集标识。rename_map 把数据集的 front/wrist 相机键映射到策略期望的 camera1/camera2；它只重命名观测键，不改变动作。batch_size=4、num_workers=4 控制显存与数据加载；env_eval_freq=0 关闭训练期间的仿真评估；W&B 和 Hub 推送均关闭，因此结果只保存在 TRAIN_OUT。

原始命令（保持不变）

```
#训练
TRAIN_OUT=/root/autodl-tmp/checkpoints/smolvla_piper_test

echo "TRAIN_OUT=$TRAIN_OUT"

lerobot-train \
```

```

--dataset.root="$PIPER_DATASET" \
--dataset.repo_id=piper_record_test \
--policy.path="$SMOLVLA_BASE" \
--policy.device=cuda \
--policy.empty_cameras=1 \
--
rename_map='{ "observation.images.front": "observation.images.camera1", "observation.images.wrist": "observation.images.camera2" }' \
--output_dir="$TRAIN_OUT" \
--job_name=smolvla_piper \
--batch_size=4 \
--num_workers=4 \
--steps=20000 \
--save_freq=5000 \
--log_freq=100 \
--env_eval_freq=0 \
--wandb.enable=false \
--policy.push_to_hub=false

```

完成标志：日志持续输出训练 loss；目录中出现 005000、010000、015000、020000 检查点，最终模型位于 020000/pretrained_model。

注意：本命令训练的是 Piper 真机模型，不是使用 HuggingFaceVLA/libero 数据集训练 LIBERO 模型。前一步 LIBERO 评估使用的仍是 SMOLVLA_LIBERO。

第五部分 真机同步与云端异步推理

下载训练检查点到机器人电脑进行同步 rollout，或通过云端 Policy Server、SSH 隧道和本地 Robot Client 运行异步推理。

步骤 20 下载检查点并运行本地同步 rollout

执行位置：机器人本地电脑；scp 源端为 AutoDL 云服务器

本步目标：通过 AutoDL SSH 端口把 20,000 步检查点复制到本地，然后以 base 策略在 Piper 上运行 20 秒同步推理。

scp 的 -r 递归复制目录，-P 38671 指定 SSH 端口。复制后的 policy.path 指向本地 pretrained_model。rollout 使用 can0、1 Mbps CAN、夹爪、弧度制和 front/wrist 两路相机；task 文本应与采集数据中的任务表达保持一致。base 策略用于直接执行，不记录新数据。

原始命令（保持不变）

```
#模型下载与本地推理
```

```

cd $HOME/lerobot_work
mkdir -p models/smolvla_piper_test_20000
scp -rP 38671 \
root@connect.westb.seetacloud.com:/root/autodl-tmp/checkpoints/smolvla_piper_test/020000/pretrained_model \
models/smolvla_piper_test_20000/
#本地推理
cd $HOME/lerobot_work
conda activate lerobot
lerobot-rollout \
--strategy.type=base \
--policy.path=./models/smolvla_piper_test_20000/pretrained_model \
--robot.type=piper \
--robot.can_interface=can0 \
--robot.bitrate=1000000 \
--robot.include_gripper=true \
--robot.use_degrees=false \
--robot.cameras="{front: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30},wrist: {type: opencv,
index_or_path: 2, width: 640, height: 480, fps: 30}}" \
--task="Put the banana on the plate" \
--duration=20

```

完成标志：本地模型目录完整，lerobot-rollout 成功连接 Piper 和两路相机，机械臂按策略执行并在 20 秒后结束。

安全提示：真机运行前先确认急停可用、机械臂周围无遮挡、CAN 接口已启动、相机索引正确。首次测试应保持人在急停附近，并从安全姿态开始。

步骤 21 在云端启动异步 Policy Server

执行位置：AutoDL 云服务器，单独终端 A

本步目标：启动只监听云端本机 127.0.0.1:8080 的策略服务，等待 Robot Client 通过握手告知策略类型、模型路径和推理设备。

Policy Server 启动时是空容器，不需要在本命令中写 policy.path。客户端首次握手后，服务器会按客户端参数加载 SmolVLA。host=127.0.0.1 不直接暴露公网端口，后续依靠 SSH 隧道转发；fps=15 是该异步链路的目标频率设置。

原始命令（保持不变）

```

#云端异步推理云端命令
conda activate lerobot

export HF_HUB_OFFLINE=1
export TRANSFORMERS_OFFLINE=1
export HF_DATASETS_OFFLINE=1

```



```
export TOKENIZERS_PARALLELISM=false

python -m lerobot.async_inference.policy_server \
  --host=127.0.0.1 \
  --port=8080 \
  --fps=15
```

完成标志：服务保持运行并监听 127.0.0.1:8080，没有端口占用或模块导入错误。

步骤 22 建立本地到云端的 SSH 隧道

执行位置：机器人本地电脑，单独终端 B

本步目标：把本地 8080 端口安全转发到云服务器的 127.0.0.1:8080，使 Robot Client 可以像连接本机服务一样访问云端 Policy Server。

-N 表示只做端口转发、不执行远程 shell；-L 的含义是“本地 8080 → 云端看到的 127.0.0.1:8080”。尖括号中的 AutoDL SSH 端口和主机地址是占位符，执行前必须替换为实例实际信息。命令成功时通常不输出内容，并会持续占用当前终端。

原始命令（保持不变）

```
#本地命令 1 通过 ssh 连接
ssh -N \
  -L 8080:127.0.0.1:8080 \
  -p <AutoDL_SSH 端口> \
  root@<AutoDL 主机地址>
例子：ssh -CNg -L 8080:127.0.0.1:8080 root@region-9.autodl.pro -p 37237
```

完成标志：SSH 连接保持不退出、没有 Connection refused；本地 8080 已被隧道占用并指向云端服务。

步骤 23 启动本地 Robot Client 进行云端异步推理

执行位置：机器人本地电脑，单独终端 C；终端 A、B 必须继续运行

本步目标：本地采集 Piper 状态和两路图像，经 SSH 隧道发送到云端 GPU；云端返回动作块，本地动作队列中连续执行并对重叠动作加权融合。

camera1/camera2 已直接使用策略期望的键名。pretrained_name_or_path 是服务器端可访问的云端路径；policy_device=cuda 指云端推理设备，client_device=cpu 指本地客户端处理设备。

actions_per_chunk=50 在 15 Hz 下名义覆盖约 3.33 秒动作；chunk_size_threshold=0.5 表示队列降到约一半时请求新动作块；weighted_average 融合重叠区间；队列可视化用于观察是否接近耗尽。

原始命令（保持不变）

#本地客户端 2

```
conda activate lerobot
```

```
python -m lerobot.async_inference.robot_client \
  --server_address=127.0.0.1:8080 \
  --robot.type=piper \
  --robot.can_interface=can0 \
  --robot.bitrates=1000000 \
  --robot.include_gripper=true \
  --robot.use_degrees=false \
  --robot.cameras='{camera1: {type: opencv, index_or_path: /dev/video6, width: 640, height: 480, fps: 10}, camera2: {type: opencv, index_or_path: /dev/video5, width: 640, height: 480, fps: 10}}' \
  --task="Put the banana on the plate." \
  --policy_type=smolvla \
  --pretrained_name_or_path=/root/autodl-tmp/checkpoints/smolvla_piper_test/020000/pretrained_model \
  --policy_device=cuda \
  --client_device=cpu \
  --fps=15 \
  --actions_per_chunk=50 \
  --chunk_size_threshold=0.5 \
  --aggregate_fn_name=weighted_average \
  --debug_visualize_queue_size=True
```

完成标志：客户端与服务器握手成功，云端加载指定检查点，动作队列持续更新，机械臂运动期间队列不应反复降到 0。

完成后的核对清单

- 环境：LeRobot 显示 0.6.0，关键依赖均已安装，pip check 无冲突。
- GPU/EGL：CUDA 可用，GPU 名称正确，EGL 探测能找到可运行设备。
- 离线资源：smolvla_libero、smolvla_base、SmolVLM2 和 LIBERO assets 均在本地完整存在。
- LIBERO：软链接指向正确 assets；四套件评估可生成日志、视频和 pc_success 成功率。
- 训练：Piper 数据集 info.json 存在；20,000 步检查点保存到预期目录。
- 同步真机：本地检查点、CAN、夹爪、相机索引和任务文本均匹配。
- 异步真机：Policy Server、SSH 隧道、Robot Client 三个终端同时在线，动作队列不反复耗尽。

关键参数速读

- `--rename_map`: 方向始终是“数据集/环境提供的源键 → 策略期望的目标键”；只影响观测键，不改动作键。
- `--policy.empty_cameras=1`: 增加一个被掩蔽的占位视觉输入，解决真实相机数量少于策略视觉槽位的问题。
- `--eval.n_episodes=10`: LIBERO 中是每个任务 10 回合；四个 10 任务套件合计 400 回合。
- `--env.control_mode=relative`: 动作按相对增量解释，必须与策略训练时的动作表示一致。
- `--actions_per_chunk=50`: 一次预测 50 步动作；在 15 Hz 下名义时长约 3.33 秒。
- `--chunk_size_threshold=0.5`: 动作队列下降到约一半时请求下一块，在连续性与更新频率之间折中。
- `--aggregate_fn_name=weighted_average`: 对新旧动作块重叠部分做加权融合，减少切块边界的突变。

参考依据

说明文字依据 LeRobot v0.6.0 官方文档及原始命令上下文整理；命令本体来源于用户上传的 `smolvla.docx`，未作修改。

SmolVLA (LeRobot v0.6.0) : <https://huggingface.co/docs/lerobot/v0.6.0/smolvla>

LIBERO (LeRobot v0.6.0) : <https://huggingface.co/docs/lerobot/v0.6.0/libero>

Rename Map and Empty Cameras: https://huggingface.co/docs/lerobot/v0.6.0/rename_map

Asynchronous Inference: <https://huggingface.co/docs/lerobot/v0.6.0/async>

Policy Deployment / lerobot-rollout: <https://huggingface.co/docs/lerobot/v0.6.0/inference>